SIMATS ENGINEERING

JAVA PROGRAMMING

ASSIGNMENT – 2

Java Strings and Collection Framework

OUTPUT

Student Details	Information
Student Name	Dhanuja D
Register Number	192311427
Department	Computer Science and Engineering
Institution	SIMATS Engineering
Course	Java Programming
Assignment	Assignment 2 – CO2
Total Marks	50
Submission Date	02/11/26

CODE:

Q1 – TicketTextBenchmark.java

```
public class TicketTextBenchmark {
static String buildString(int n) {
String s = "";
for (int i = 1; i <= n; i++) {
s = s + "Field" + i + ":Value" + i + "\n";
}
return s;
}
static String buildBuilder(int n) {
StringBuilder s = new StringBuilder();
for (int i = 1; i <= n; i++) {
s.append("Field").append(i).append(":Value").append(i).append("\n");
}
return s.toString();
}
static String buildBuffer(int n) {
StringBuffer s = new StringBuffer();
for (int i = 1; i <= n; i++) {
s.append("Field").append(i).append(":Value").append(i).append("\n");
}
return s.toString();
```

```
}
static long measure(Runnable task, int runs) {
long start = System.nanoTime();
for (int i = 0; i < runs; i++) {
task.run();
}
return (System.nanoTime() - start) / runs;
}
public static void main(String[] args) {
int n = 50;
int runs = 2000;
System.out.println("String average ns: " + measure(() -> buildString(n), runs));
System.out.println("StringBuilder average ns: " + measure(() -> buildBuilder(n), runs));
System.out.println("StringBuffer average ns: " + measure(() -> buildBuffer(n), runs));
System.out.println("Ticket Preview:");
System.out.print(buildBuilder(5));
}
}
```

Q2 – PlaylistComparison.java

```
import java.util.*;
public class PlaylistComparison {
```

```
static long testArrayList(int n) {
ArrayList<Integer> list = new ArrayList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
list.add(i);
}
for (int i = 0; i < n; i++) {
list.get(i);
}
return System.nanoTime() - start;
}
static long testLinkedList(int n) {
LinkedList<Integer> list = new LinkedList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
list.add(i);
}
for (int i = 0; i < n; i++) {
list.get(i);
}
return System.nanoTime() - start;
}
public static void main(String[] args) {
int[] sizes = {1000, 5000, 10000};
```

```
for (int n : sizes) {
System.out.println("n = " + n);
System.out.println("ArrayList ns = " + testArrayList(n));
System.out.println("LinkedList ns = " + testLinkedList(n));
}
ArrayDeque<String> queue = new ArrayDeque<>();
queue.addLast("Song A");
queue.addLast("Song B");
queue.addFirst("Priority Song");
System.out.println("Next song: " + queue.removeFirst());
}
}
```

Q3 – AttendanceDemo.java

```
import java.util.*;
class Student {
String regNo, name;
Student(String regNo, String name) {
this.regNo = regNo;
this.name = name;
}
```

```
public boolean equals(Object obj) {
if (this == obj) return true;
if (!(obj instanceof Student)) return false;
Student s = (Student) obj;
return Objects.equals(regNo, s.regNo);
}
public int hashCode() {
return Objects.hash(regNo);
}
public String toString() {
return regNo + " " + name;
}
}
public class AttendanceDemo {
public static void main(String[] args) {
Set<Student> students = new HashSet<>();
students.add(new Student("192311001", "Arun"));
students.add(new Student("192311002", "Divya"));
students.add(new Student("192311001", "Arun"));
System.out.println("Unique students: " + students.size());
for (Student s : students) {
System.out.println(s);
}
}
```

```
}
```

Q4 – PharmacyInventory.java

```
import java.util.*;
public class PharmacyInventory {
public static void main(String[] args) {
Map<String, Integer> hash = new HashMap<>();
Map<String, Integer> tree = new TreeMap<>();
Map<String, Integer> linked = new LinkedHashMap<>();
String[] codes = {"M103", "M101", "M105", "M102", "M104"};
int[] stocks = {40, 25, 60, 30, 15};
for (int i = 0; i < codes.length; i++) {
hash.put(codes[i], stocks[i]);
tree.put(codes[i], stocks[i]);
linked.put(codes[i], stocks[i]);
}
System.out.println("Lookup M103: " + hash.get("M103"));
System.out.println("Sorted code report: " + tree);
System.out.println("Order-added report: " + linked);
System.out.println("HashMap report: " + hash);
}
}
```

Q5 – LeaderboardDemo.java

```
import java.util.*;
class Team {
String name;
int points, goalDiff;
Team(String name, int points, int goalDiff) {
this.name = name;
this.points = points;
this.goalDiff = goalDiff;
}
public String toString() {
return name + "(" + points + "," + goalDiff + ")";
}
}
class Leaderboard<T> {
List<T> items = new ArrayList<>();
Comparator<? super T> order;
Leaderboard(Comparator<? super T> order) {
this.order = order;
}
void add(T item) {
items.add(item);
}
List<T> ranking() {
List<T> result = new ArrayList<>(items);
```

```
result.sort(order);

return result;

}

}

public class LeaderboardDemo {

public static void main(String[] args) {

Comparator<Team> rank = Comparator.comparingInt((Team t) ->
t.points)

.reversed()

.thenComparing(Comparator.comparingInt((Team t) ->
t.goalDiff).reversed())

.thenComparing(t -> t.name);

Leaderboard<Team> board = new Leaderboard<>(rank);

board.add(new Team("Panthers", 9, 4));

board.add(new Team("Eagles", 9, 6));

board.add(new Team("Cobras", 6, 1));

board.add(new Team("Bisons", 9, 4));

System.out.println(board.ranking());

}

}
```

Q6 – Triage.java

```
import java.util.*;

class Patient {

String name;
```

```
int severity;
long arrival;
Patient(String name, int severity, long arrival) {
this.name = name;
this.severity = severity;
this.arrival = arrival;
}
public String toString() {
return name + "(S" + severity + ")";
}
}
public class Triage {
long counter = 0;
PriorityQueue<Patient> queue = new PriorityQueue<>(
Comparator.comparingInt((Patient p) -> p.severity)
.thenComparingLong(p -> p.arrival)
);
void admit(String name, int severity) {
if (severity < 1 || severity > 5) {
throw new IllegalArgumentException("Severity must be 1 to 5");
}
queue.add(new Patient(name, severity, counter++));
}
public static void main(String[] args) {
```

```
Triage er = new Triage();
er.admit("Ravi", 3);
er.admit("Meena", 1);
er.admit("John", 3);
er.admit("Asha", 5);
er.admit("Kumar", 1);
er.admit("Latha", 2);
Patient p;
StringBuilder result = new StringBuilder();
while ((p = er.queue.poll()) != null) {
if (result.length() > 0) result.append(" ");
result.append(p);
}
System.out.println(result);
}
}
```

Q7 – FrontOperationBenchmark.java

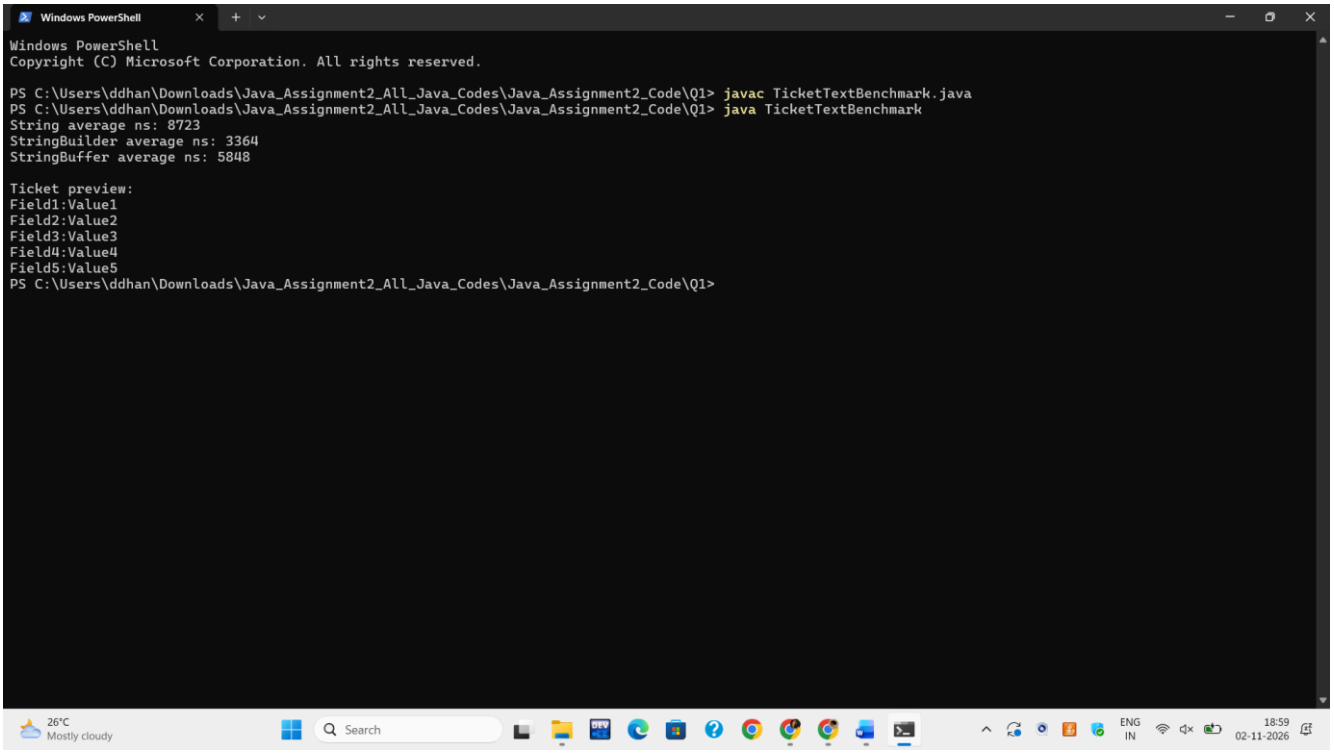
```
import java.util.*;
public class FrontOperationBenchmark {
static long testArrayList(int n) {
ArrayList<Integer> list = new ArrayList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
```

```
list.add(0, i);
}
while (!list.isEmpty()) {
list.remove(0);
}
return System.nanoTime() - start;
}
static long testArrayDeque(int n) {
ArrayDeque<Integer> deque = new ArrayDeque<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
deque.addFirst(i);
}
while (!deque.isEmpty()) {
deque.removeFirst();
}
return System.nanoTime() - start;
}
public static void main(String[] args) {
int[] sizes = {1000, 5000, 10000};
int runs = 5;
for (int n : sizes) {
testArrayList(Math.min(n, 1000));
testArrayDeque(Math.min(n, 1000));
```

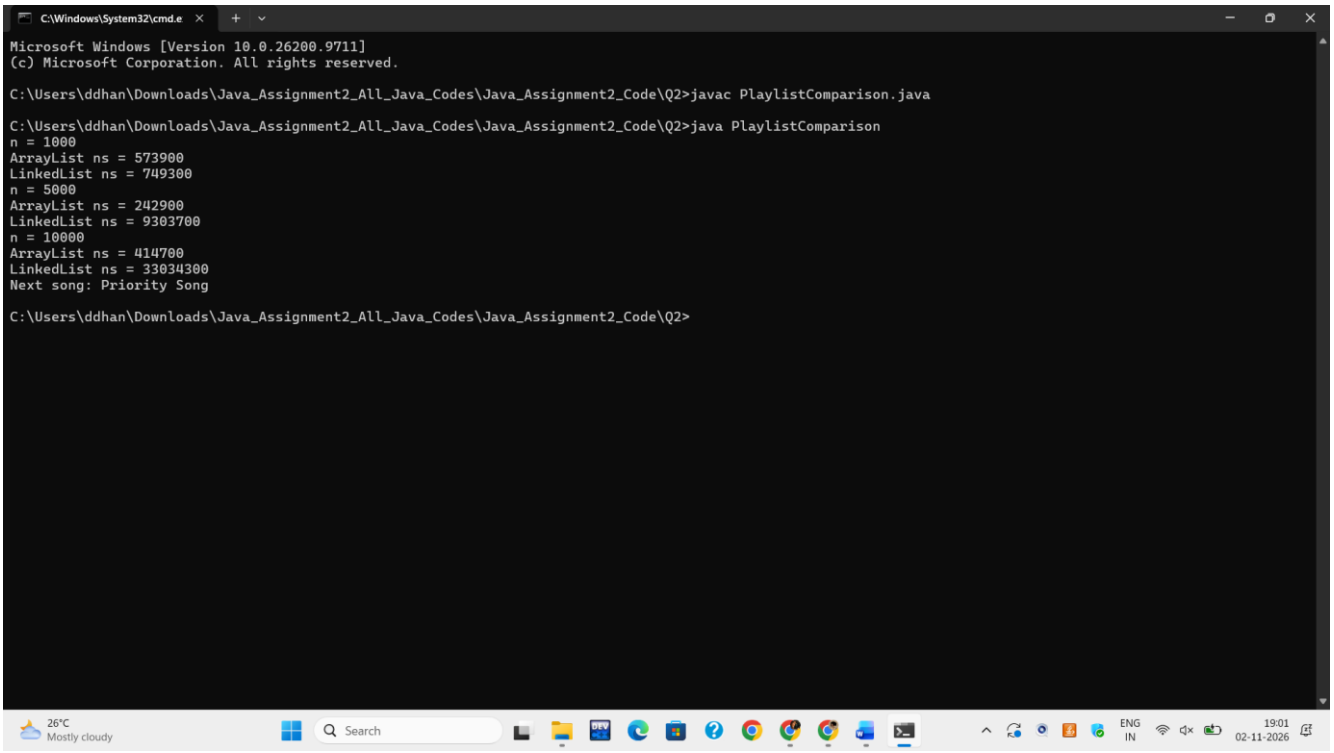
```
long a = 0, d = 0;
for (int i = 0; i < runs; i++) {
a += testArrayList(n);
d += testArrayDeque(n);
}
System.out.println("n = " + n);
System.out.println("ArrayList average ns = " + (a / runs));
System.out.println("ArrayDeque average ns = " + (d / runs));
}
}
}
```

APPENDIX A – OUTPUT

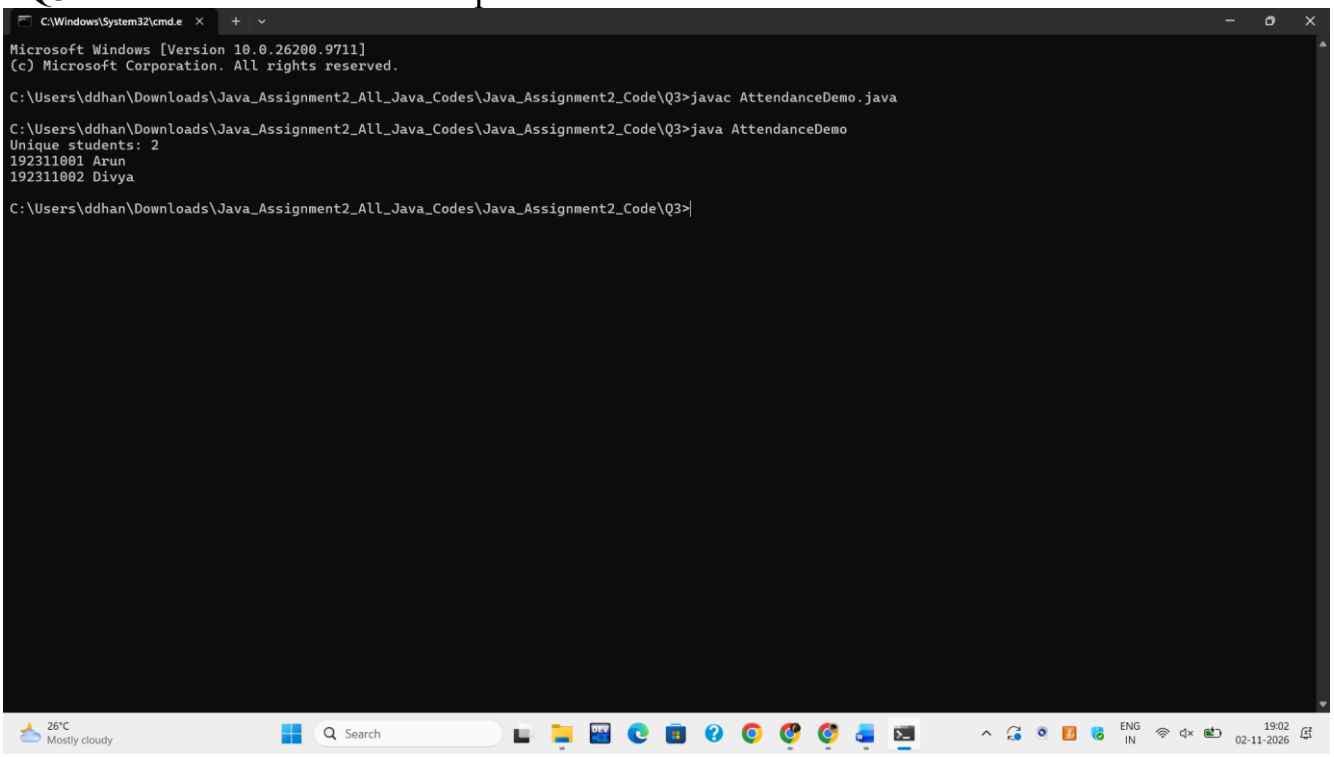
- Q1: String, StringBuilder and StringBuffer



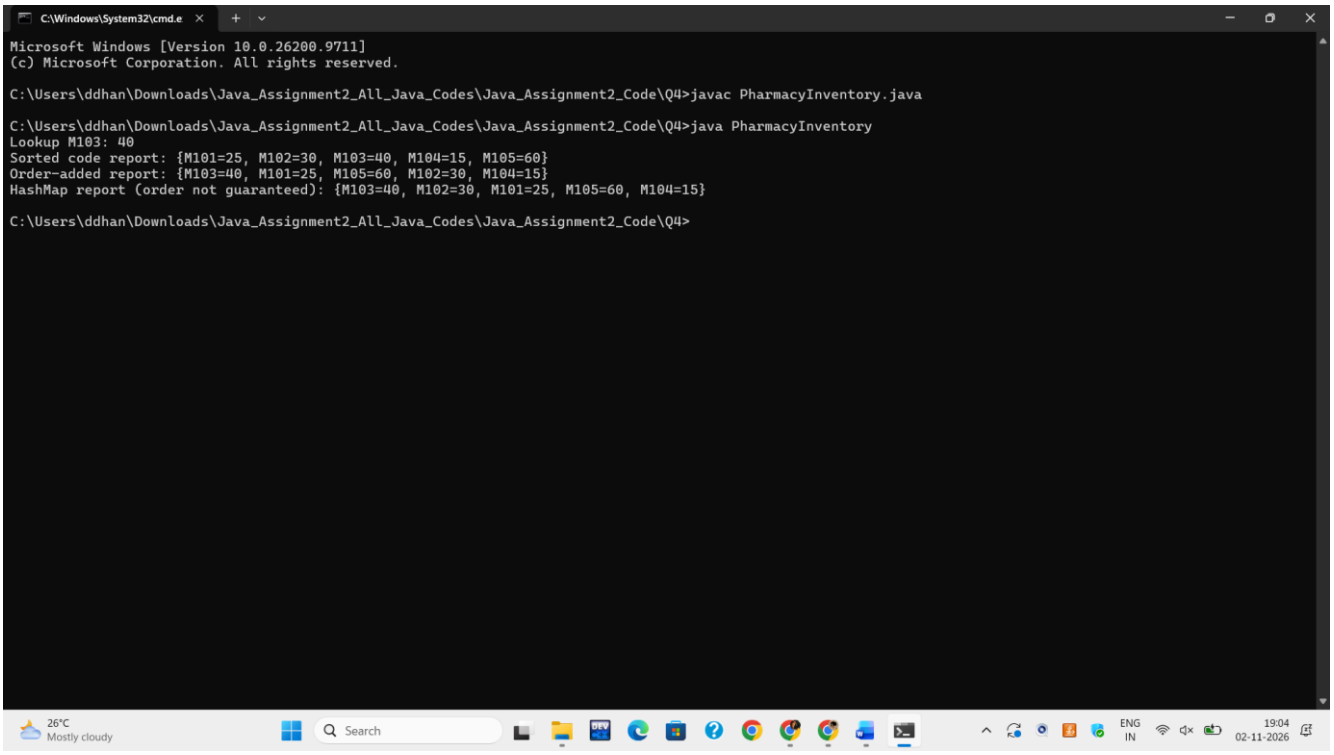
- Q2: ArrayList, LinkedList and ArrayDeque



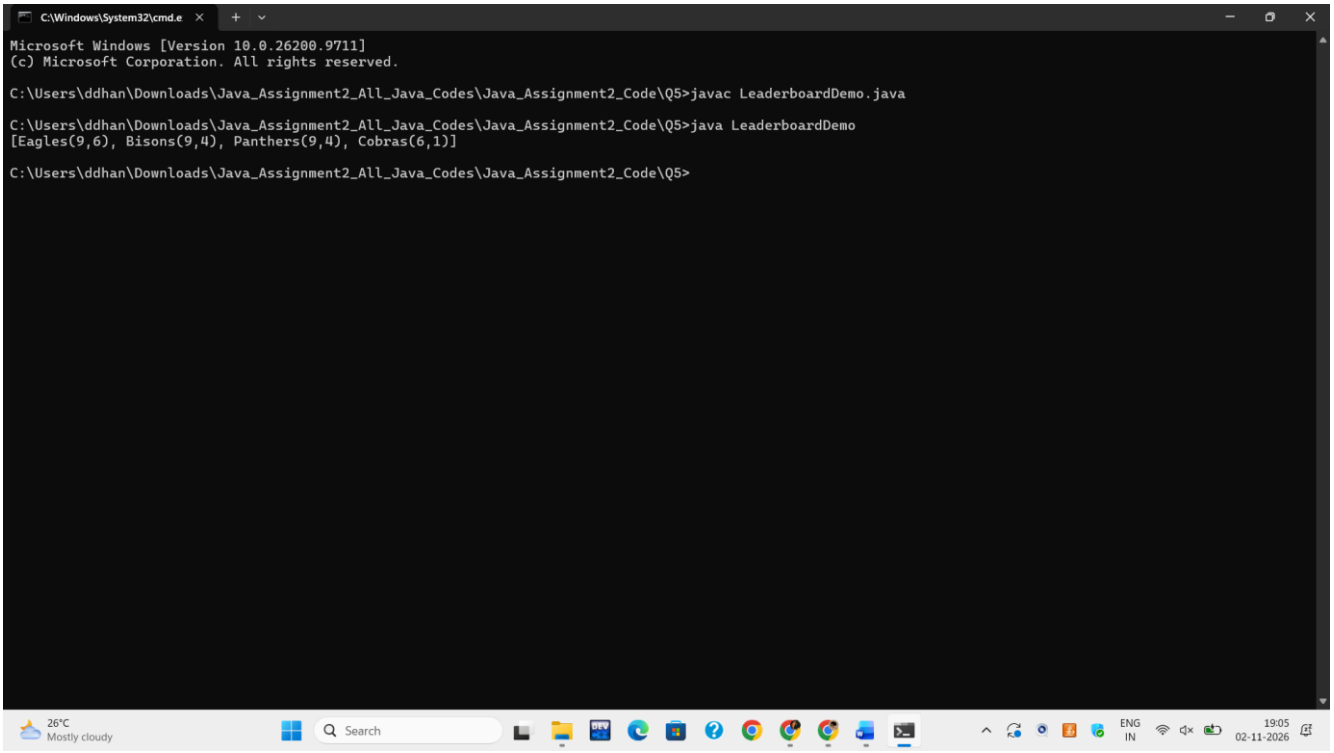
- Q3: HashSet and hashCode-equals



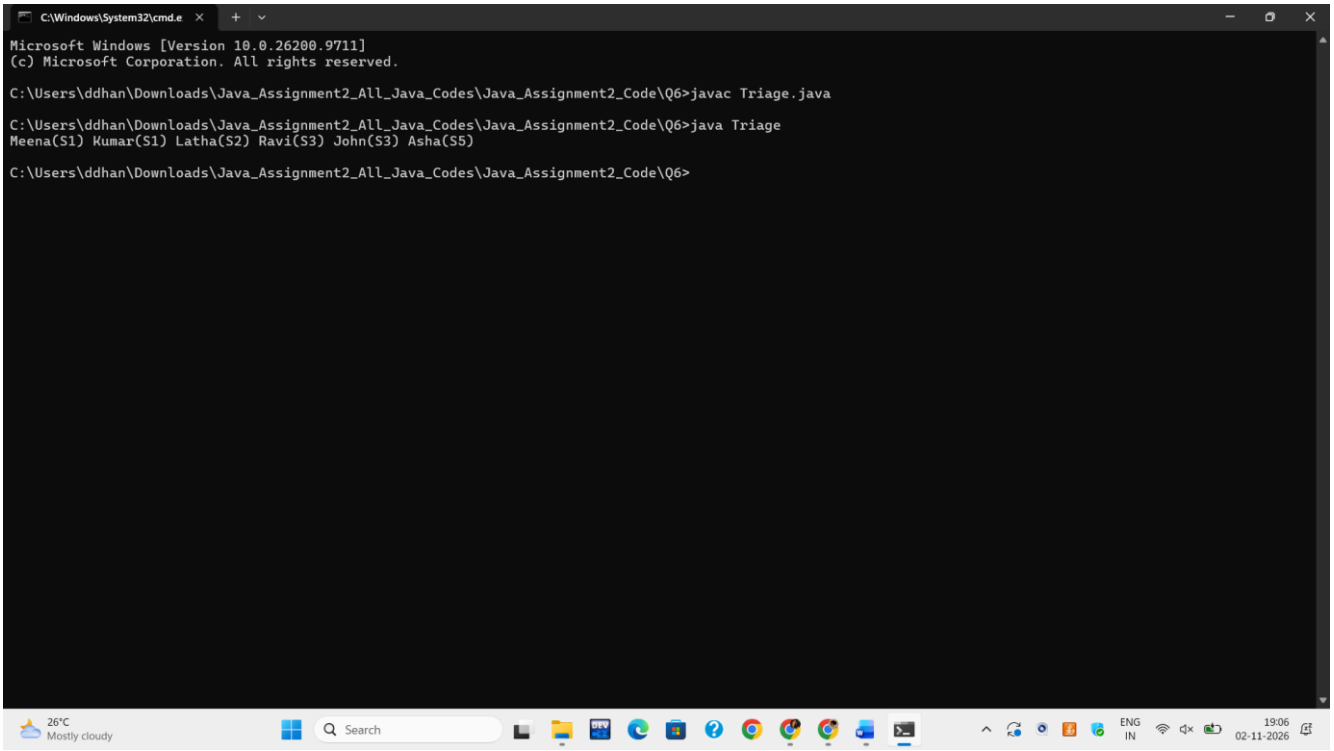
- Q4: HashMap, TreeMap and LinkedHashMap



- Q5: Generics, Comparable and Comparator



- Q6: Queue and PriorityQueue



- Q7: Diagnose, redesign and validate

